



**U N I V E R S I D A D
D E L A F R O N T E R A**

Pruebas de software

Implementación de pruebas de Carga y Estrés sobre un Sistema de Gestión de Juegos

Integrantes:

- Diego López
- Ignacio Uribe

Arquitectura de la infraestructura de pruebas

La infraestructura se compone de dos stacks Docker independientes pero conectados mediante una red Docker común (prueba-carga-network). El stack principal contiene la API y la base de datos. El stack de monitoreo contiene:

- Prometheus (v2.54.1): almacenamiento de series temporales con endpoint de `remote_write` habilitado para recibir métricas de K6.
- `postgres_exporter` (v0.15.0): expone métricas internas de PostgreSQL (conexiones activas, transacciones por segundo, locks).
- `cAdvisor` (v0.49.1): expone métricas de uso de recursos (CPU, memoria, red) a nivel de contenedor.
- Grafana (v11.2.0): visualización de métricas mediante un dashboard auto-provisionado con paneles para métricas K6, PostgreSQL y `cAdvisor`.
- K6 (v0.53.0): invocado bajo demanda mediante Docker Compose profiles, no se ejecuta como servicio permanente.

Diseño de las pruebas

Cada usuario virtual (VU) ejecuta en cada iteración el ciclo de vida completo del recurso: POST para crear un juego, GET para listar todos, GET para leer el recurso recién creado, PUT para actualizarlo y DELETE para eliminarlo. Este diseño garantiza que la prueba ejerce los cinco endpoints en proporción equilibrada y que la base de datos no acumula registros huérfanos entre iteraciones.

Los payloads de los requests POST y PUT se parametrizan mediante un archivo CSV (`games.csv`) con 20 juegos reales, garantizando variabilidad en los datos enviados.

Configuración de la prueba de carga

La prueba de carga (`plan-carga.js`) busca verificar el comportamiento del sistema bajo la carga objetivo definida en el enunciado:

- Ramp-up: 30 segundos para pasar de 0 a 100 VUs.
- Carga sostenida: 10 minutos manteniendo 100 VUs activos.
- Ramp-down: 30 segundos para descender de 100 a 0 VUs.
- Criterios de aceptación (thresholds): error rate inferior al 5%, latencia p95 global inferior a 2000 ms, latencia p95 de lectura inferior a 500 ms y latencia p95 de escritura inferior a 1500 ms.

Configuración de la prueba de estrés

La prueba de estrés (plan-estrés.js) busca identificar el punto de quiebre del sistema mediante un incremento progresivo de la carga en escalones de dos minutos:

Escalón	VUs concurrentes	Propósito
1	50	Baseline cargado
2	100	Equivalente a la prueba de carga
3	200	Doble de la carga objetivo
4	400	Presión sobre el pool de conexiones
5	600	Saturación esperada
6	800	Punto de quiebre esperado
7	1000	Límite superior

A diferencia de la prueba de carga, el script de estrés implementa un lifecycle tolerante a fallos: cada paso del CRUD válida la respuesta del paso anterior antes de continuar, lo que evita cascadas de errores cuando la API se encuentra saturada. Los thresholds estan configurados como informativos (abortOnFail: false), permitiendo que la prueba complete los 18 minutos previstos sin detenerse aun cuando se observen tasas de error elevadas. Adicionalmente, cada request tiene un timeout de 15 segundos para evitar que los VUs queden bloqueados indefinidamente.

Criterios de quiebre

Para identificar el punto de quiebre del sistema durante la prueba de estrés se establecieron cuatro criterios. La ocurrencia sostenida durante 30 segundos de cualquiera de ellos califica como quiebre:

- Tasa de error superior al 5% (HTTP 5xx o timeouts).
- Latencia p95 superior a 2000 ms.
- Caída del throughput mientras los VUs siguen incrementándose (saturación).
- Errores de conexión del cliente hacia la API (ECONNREFUSED, request timeout).

Resultados de la prueba de carga

La prueba de carga se ejecutó el 15 de junio de 2026 entre las 02:09 y las 02:21 hora local, alcanzando los 100 VUs objetivo durante un período sostenido de 10 minutos. Los resultados muestran un comportamiento estable del sistema, sin errores reportados y con latencias muy por debajo de los thresholds definidos.

Métricas globales obtenidas

Métrica	Valor obtenido
VUs concurrentes sostenidos	100
Duración total	11 minutos (incl. ramp-up/down)
Total de requests procesados	84.800
Iteraciones CRUD completas	28.500
Throughput sostenido	~45 req/s
Latencia p95 (PUT, más lento)	~160 ms
Latencia p95 (GET por id, más rápido)	~5 ms
Tasa de error HTTP	0,00 %
Transacciones SQL sostenidas	~230 ops/s

Vista general del dashboard durante la prueba

La figura 1 muestra la vista consolidada del dashboard durante la ejecución. Se distinguen cinco bloques: métricas K6, indicadores agregados, métricas de PostgreSQL y métricas de contenedores. El período de carga sostenida se visualiza como una meseta plana sin oscilaciones.



Figura 1: Vista general del dashboard durante la prueba de carga.

Latencia por endpoint

La figura 2 muestra la evolución del percentil 95 de latencia desagregada por verbo HTTP. Se observa una rampa de calentamiento entre las 02:09 y las 02:14, seguida de una meseta plana entre las 02:14 y las 02:19 durante el período de máxima concurrencia. La latencia más elevada corresponde a PUT_games (160 ms), seguida de POST_games (140 ms) y GET_games_list (138 ms). El endpoint GET_games_by_id resultó el más rápido (5 ms) gracias a la consulta por clave primaria indexada.



Figura 2: Latencia p95 por endpoint durante la prueba de carga.

Throughput sostenido

La figura 3 muestra el throughput agrupado por endpoint. La meseta plana en torno a los 45 req/s sostenidos durante 10 minutos sin caídas ni picos espurios constituye la evidencia visual más directa de que el sistema cumple holgadamente con la carga objetivo del enunciado.

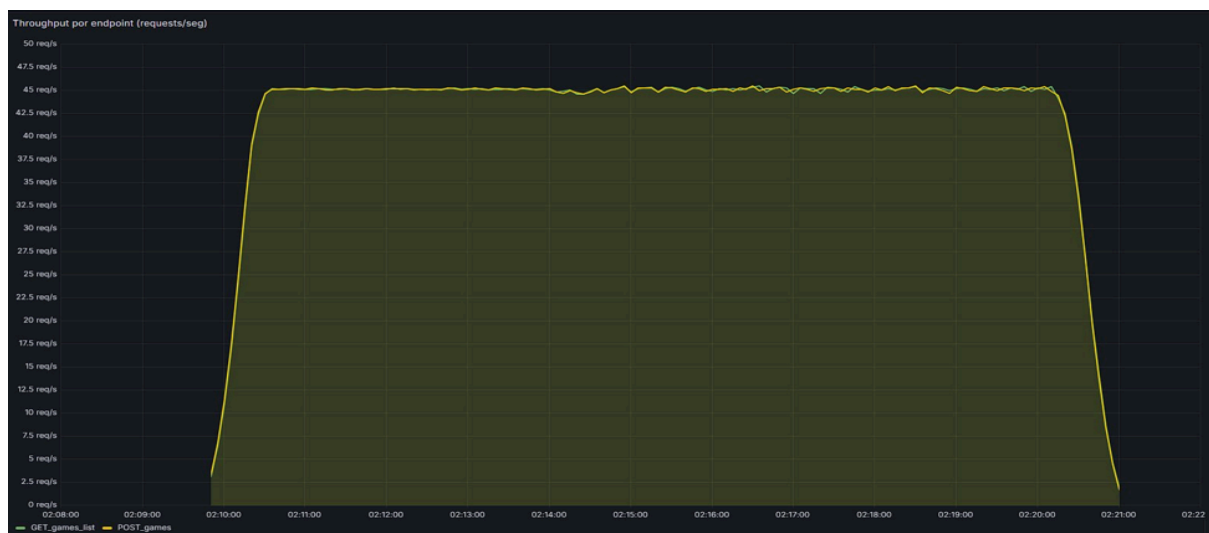


Figura 3: Throughput por endpoint durante la prueba de carga.

Indicadores agregados

La figura 4 consolida los indicadores instantáneos al cierre de la prueba: un total de 84.800 requests procesados, 28.500 iteraciones CRUD completas y 0 % de tasa de error. La meseta visible en el sparkline de VUs activos refleja el sostenimiento de los 100 usuarios virtuales durante los 10 minutos centrales.

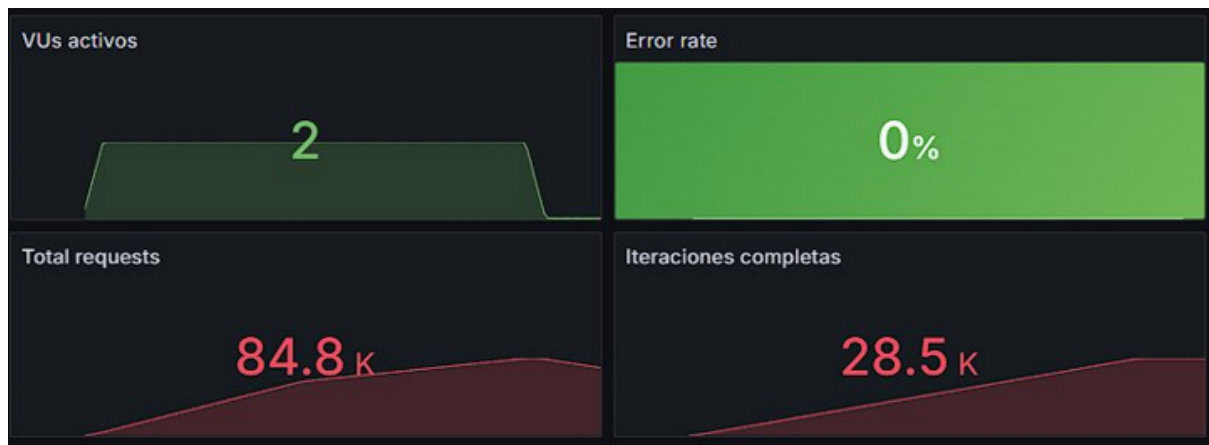


Figura 4: Indicadores agregados al cierre de la prueba de carga.

Comportamiento de PostgreSQL

Las figuras 5 y 6 muestran el comportamiento de la base de datos durante la prueba. El panel de conexiones revela que el pool por defecto de 10 conexiones del driver pg estuvo saturado durante toda la prueba: la línea anaranjada que representa conexiones idle del pool oscila entre 8 y 11. Pese a esta saturación, el sistema mantuvo un throughput estable de aproximadamente 230 transacciones por segundo y rollbacks plano en cero, indicando que la BD respondió adecuadamente a la carga generada.



Figura 5: Conexiones activas vs estado del pool durante la prueba de carga.



Figura 6: Transacciones SQL por segundo durante la prueba de carga.

Conclusión parcial de la prueba de carga

La API sostuvo los 100 VUs objetivo durante 10 minutos sin reportar errores HTTP, con latencias p95 inferiores a 200 ms en todos los endpoints. La saturación del pool de conexiones observada sugiere que el sistema se encuentra cerca del límite de capacidad con esta configuración, hipótesis que se valida formalmente en la prueba de estrés.

Resultados de la prueba de estrés

La prueba de estrés se ejecutó el 15 de junio de 2026 entre las 20:45 y las 21:03 hora local, con una duración total efectiva de 18 minutos 18 segundos. Durante esta prueba se observó claramente el punto de quiebre del sistema, lo que permite responder con precisión a la solicitud del enunciado de identificar el umbral de degradación.

Métricas globales obtenidas

Métrica			Valor obtenido
VUs máximo configurados			1.000
VUs máximo efectivamente sostenidos			600 (colapso antes de alcanzar 800)
Total de requests procesados			327.686
Iteraciones CRUD completas			65.616
Iteraciones abortadas (lifecycle interrumpido)			753

Throughput pico de la API (cliente)	240 req/s
Throughput promedio de la API	298 req/s
Transacciones SQL pico	1.100 ops/s
Latencia p95 máxima por endpoint	30 segundos (POST_games)
Tasa de error HTTP	0,41 % (1.356 fallos)
Tasa global de éxito CRUD	99,08 %

Vista general del dashboard

La figura 7 muestra la vista consolidada del dashboard durante la ejecución de la prueba de estrés. A diferencia de la prueba de carga, en este caso se observa un evento claro de saturación alrededor de las 20:54, visible como un salto vertical en los paneles de latencia y como una caída abrupta en el panel de throughput.

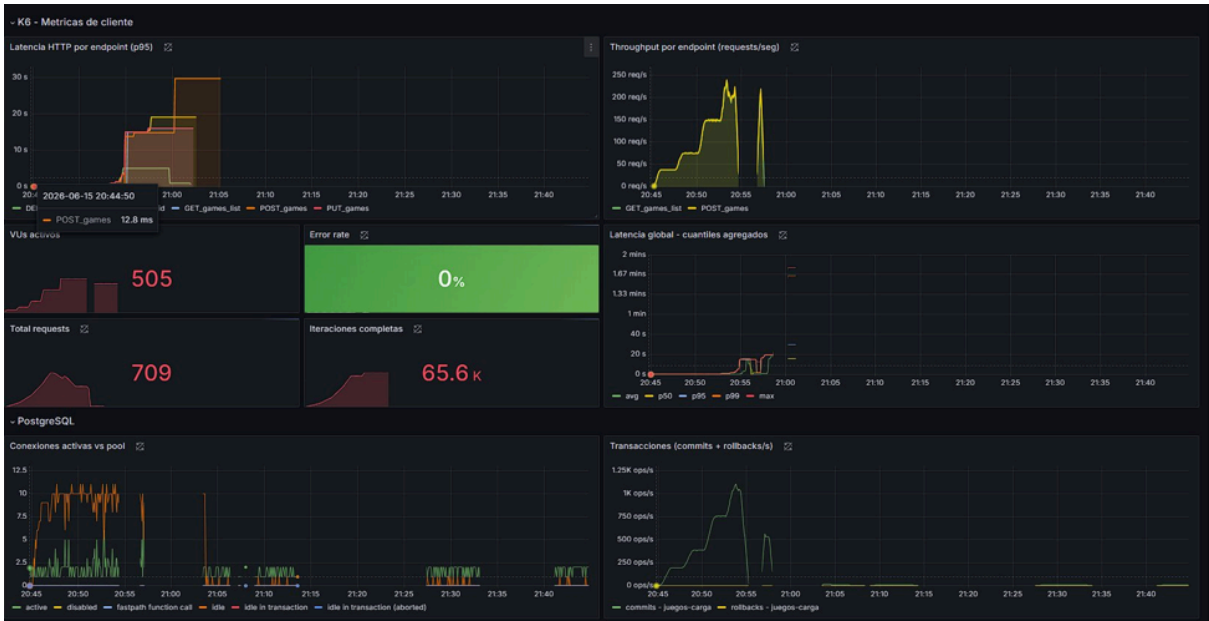


Figura 7: Vista general del dashboard durante la prueba de estrés.

Diseño de escalones aplicado (VUs)

La figura 8 confirma la aplicación correcta del diseño de escalones: los primeros cinco niveles (50, 100, 200, 400 y 600 VUs) son claramente visibles como mesetas planas de dos minutos cada una. Sin embargo, la curva no alcanza los escalones planificados de 800 y 1000 VUs: tras llegar a 600 VUs alrededor de las 21:00, se observa una caída sostenida

hasta 505 VUs, evidencia de que K6 no logró mantener el conteo de usuarios virtuales activos debido a que las iteraciones quedaban colgadas por encima del timeout configurado de 15 segundos.

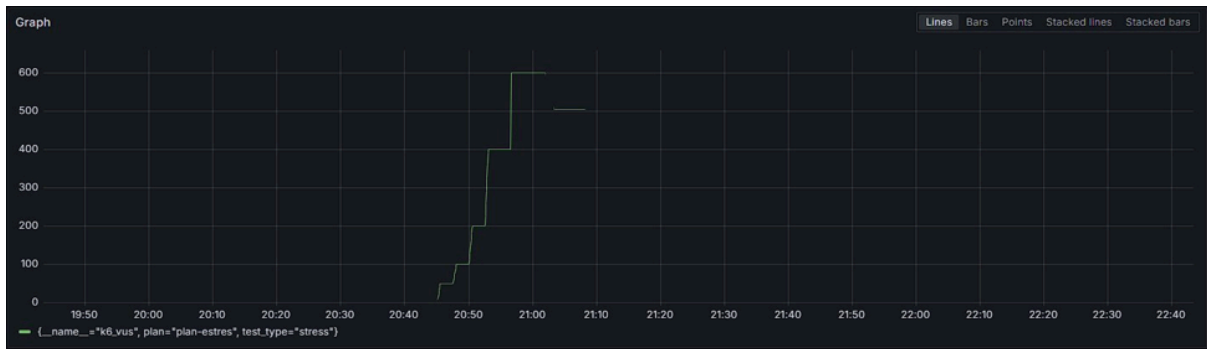


Figura 8: Evolución de VUs activos durante la prueba de estrés.

Identificación del punto de quiebre

La figura 9 constituye la evidencia central del informe. Muestra la evolución de la latencia p95 desagregada por endpoint a lo largo de toda la prueba. Se observan tres regiones diferenciadas:

- **Región estable (20:40 - 20:54):** durante los escalones 1, 2 y 3 (50, 100 y 200 VUs) la latencia se mantiene por debajo de los 5 segundos, con rampas suaves entre escalones.
- **Salto vertical (20:54 - 20:55):** al iniciar el escalón 4 (400 VUs) la latencia salta de aproximadamente 5 segundos a 15 segundos en cuestión de segundos. Este es el punto de quiebre del sistema.
- **Región de saturación progresiva (20:55 - 21:00):** a partir del quiebre la latencia continúa escalando hasta alcanzar los 30 segundos (límite del timeout configurado), confirmando que el sistema queda funcionalmente inutilizable bajo cargas superiores.

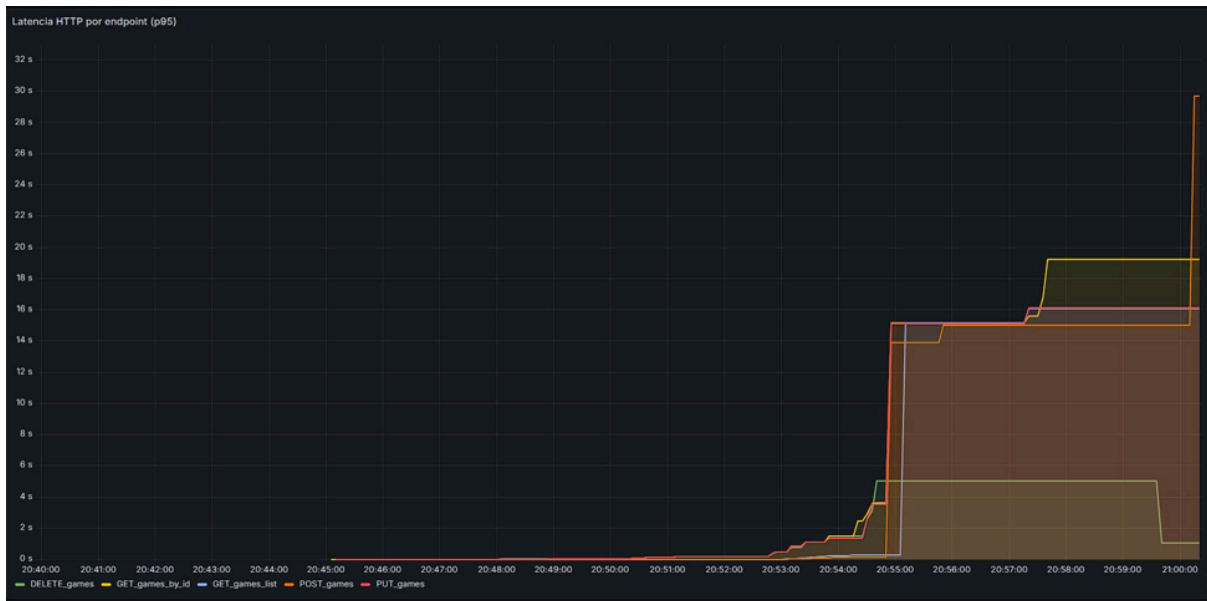


Figura 9: Latencia p95 por endpoint - identificación del punto de quiebre.

La figura 10 muestra los mismos cuantiles pero agregados globalmente (no por endpoint). Se observa el mismo patrón: avg, p50, p95 y p99 trepan conjuntamente desde valores cercanos a cero hasta los 20 segundos durante el evento de quiebre. El máximo absoluto registrado (línea aislada en 1,83 minutos, aproximadamente 110 segundos) corresponde a iteraciones cuyas requests sufrieron timeouts y retries internos del lifecycle

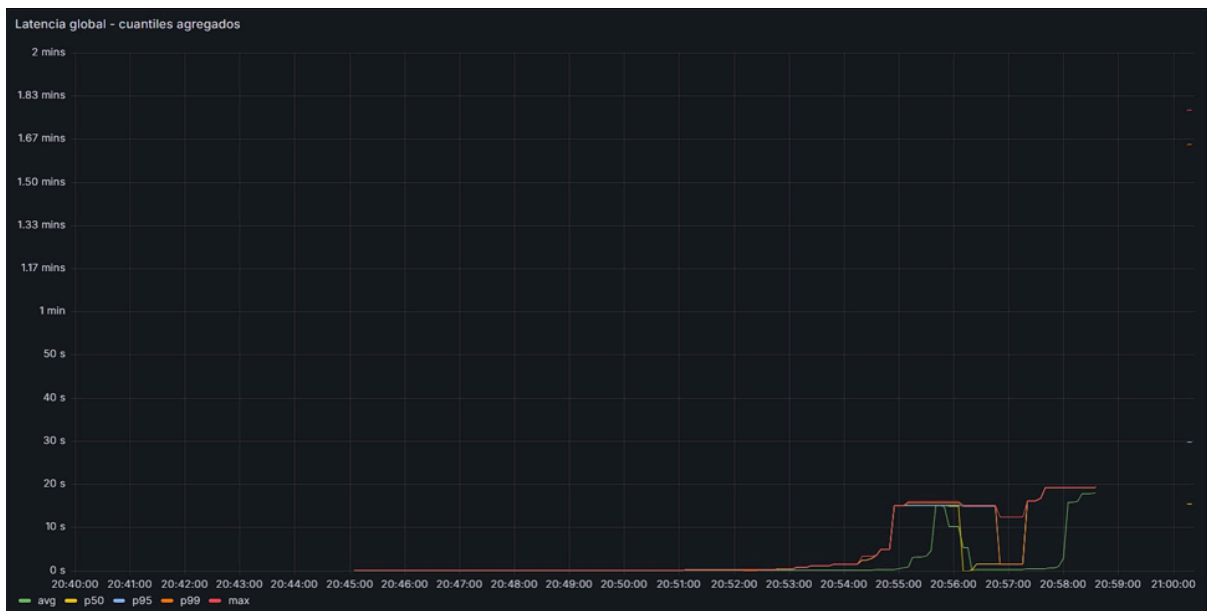


Figura 10: Cuantiles agregados de latencia durante la prueba de estrés.

Colapso del throughput

La figura 11 muestra cómo, mientras la carga (VUs) seguía aumentando, el throughput de la API alcanzó un pico de aproximadamente 240 req/s alrededor de las 20:53 y comenzó a

desplomarse hacia cero. Este patrón es la firma clásica de saturación conocida como "knee" en la curva de respuesta de sistemas bajo carga: superado el punto de equilibrio entre demanda y capacidad, el sistema procesa menos trabajo en lugar de más.

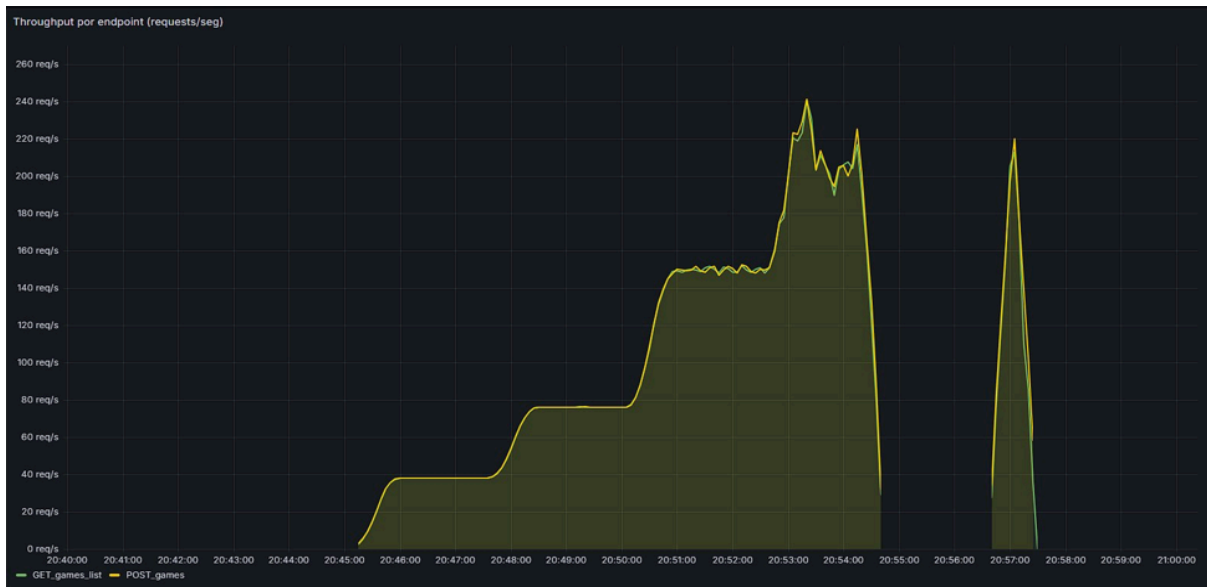


Figura 11: Colapso del throughput por endpoint en la prueba de estrés.

Indicadores agregados al final de la prueba



Figura 12: Indicadores agregados al cierre de la prueba de estrés.

La figura 12 consolida los indicadores finales: 600 VUs activos en el momento de la captura, 0 % de error rate HTTP visible (cifra que requiere matización: se discute en la sección Análisis de cuellos de botella), 149.000 requests totales acumulados en el período visible y 65.600 iteraciones CRUD completas.

Análisis de cuellos de botella

La correlación entre las métricas del lado cliente (K6) y del lado servidor (postgres_exporter y cAdvisor) permite identificar con precisión la causa raíz de la degradación del sistema y descartar otras hipótesis.

Saturación del pool de conexiones

La figura 13 muestra la evolución de las conexiones activas hacia PostgreSQL durante la prueba de estrés. Desde el inicio del escalón 1 (50 VUs) el pool de 10 conexiones del driver pg ya está operativo en su límite práctico (8 a 11 conexiones simultáneas), oscilando entre estados active e idle in transaction. Esta saturación se mantiene a lo largo de toda la prueba, lo que demuestra que el pool ya estaba en su tope con 50 VUs y no podía escalar para atender más carga.

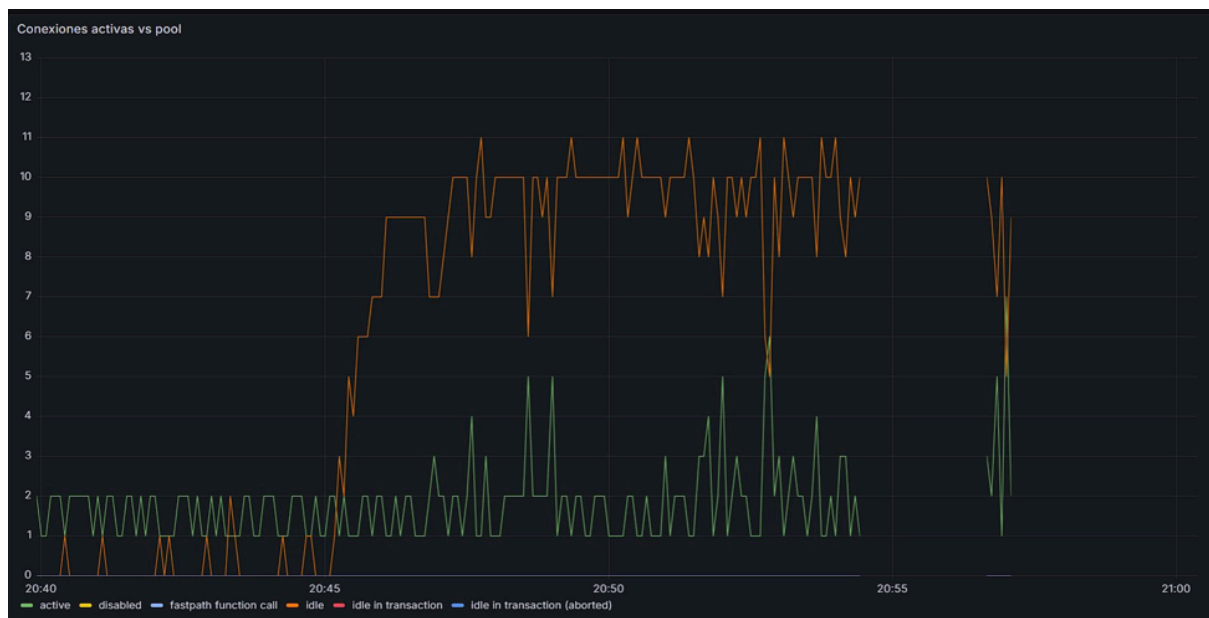


Figura 13: Conexiones activas vs estado del pool durante la prueba de estrés.

Transacciones SQL

La figura 14 muestra el throughput de transacciones SQL completadas durante la prueba. Se observa una curva en forma de S: rampa progresiva hasta un pico de aproximadamente 1.100 ops/s alrededor de las 20:53-20:54, seguida de una caída abrupta a cero coincidente con el colapso del lado cliente. El indicador clave es que la línea de rollbacks permanece plana en cero durante toda la prueba: la base de datos no rechazó transacciones, simplemente no recibió más trabajo porque las conexiones estaban encoladas esperando turno en el pool.

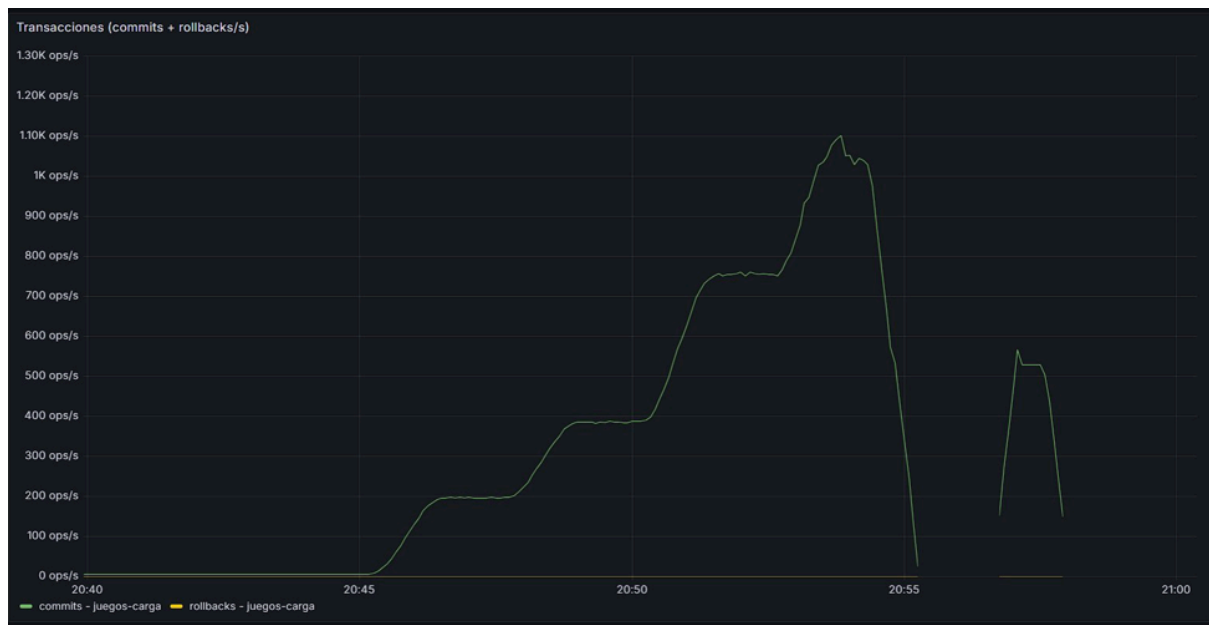


Figura 14: Transacciones SQL por segundo durante la prueba de estrés.

Uso de CPU por contenedor

La figura 15 muestra el uso de CPU diferenciado por contenedor durante la prueba. La API (línea verde) alcanza un pico cercano a 0,95 cores hacia las 20:54, mientras que PostgreSQL (línea amarilla) se mantiene en torno a 0,4 cores en su pico máximo. Esta diferencia es determinante para el diagnóstico: la base de datos no era el cuello de botella desde el punto de vista de CPU, pues operaba con holgura cercana al 60 % de capacidad disponible. La API consumía más CPU porque debía gestionar la cola de requests bloqueadas esperando una conexión libre del pool.

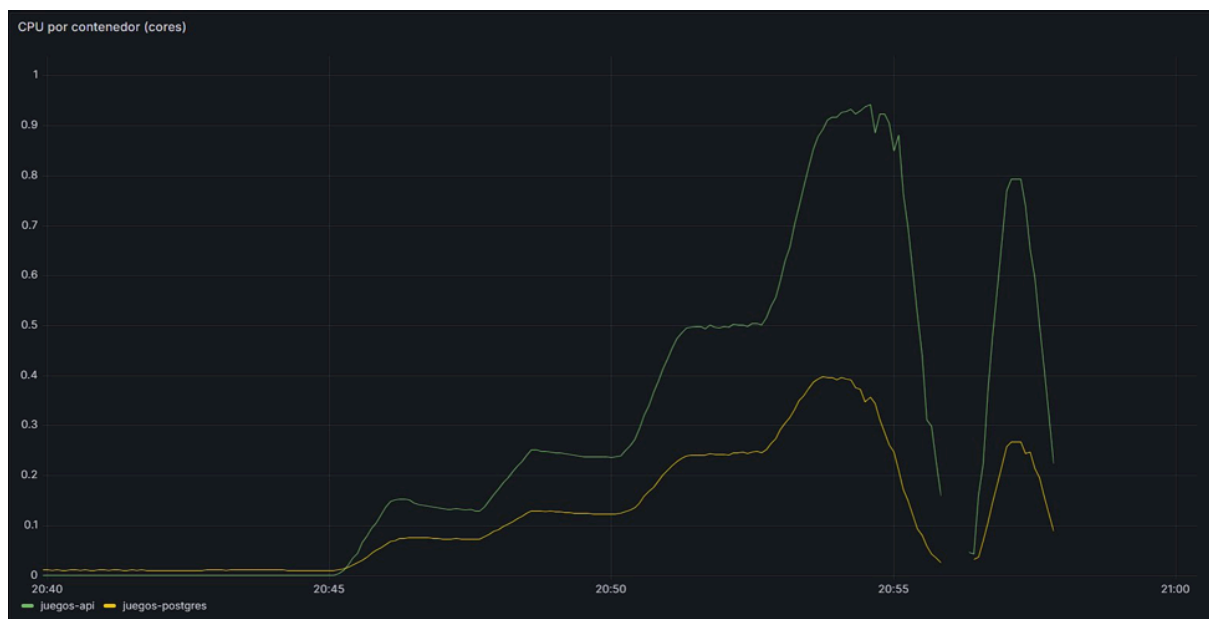


Figura 15: Uso de CPU por contenedor durante la prueba de estrés.

Uso de memoria por contenedor

La figura 16 muestra la evolución del consumo de memoria de ambos contenedores. La API (verde) parte de un baseline de 16 MiB en estado idle, sube progresivamente a 64 MiB durante los escalones iniciales y experimenta un crecimiento agresivo hasta aproximadamente 165 MiB durante el período de saturación (20:53-20:55). Tras el ramp-down la memoria no retorna al baseline inicial, sino que se estabiliza en torno a 160 MiB, comportamiento que sugiere la posible presencia de una fuga de memoria menor relacionada con los buffers de requests encoladas. PostgreSQL (amarillo) muestra un patrón mucho más saludable: crecimiento moderado durante la carga y liberación correcta al terminar la prueba.



Figura 16: Uso de memoria por contenedor durante la prueba de estrés.

Síntesis del diagnóstico

La evidencia recogida permite concluir que el cuello de botella principal del sistema no es de origen hardware ni de capacidad de la base de datos, sino de configuración del pool de conexiones de la aplicación. Postgres dispuso de más del 60 % de su CPU disponible y de memoria controlada durante todo el experimento. La API, por su parte, consumió casi un core completo gestionando la cola de espera de requests que no podían acceder a una conexión. Ningún síntoma observado se explica si no se asume la limitación del pool: la saturación temprana, la caída del throughput, la explosión de la latencia, el crecimiento de memoria de la API y el bajo uso de CPU de PostgreSQL son consistentes con un sistema saturado en su capa de coordinación (pool) antes que en su capa de ejecución (CPU/IO de BD).

Adicionalmente, la observación de que la latencia más alta corresponde sistemáticamente al endpoint GET /api/games (listado completo sin paginación) sugiere un segundo cuello de botella secundario: el costo de serializar y transmitir respuestas que crecen linealmente con el número de registros almacenados.

Conclusiones y recomendaciones

El ejercicio cumplió en su totalidad con los entregables solicitados en el enunciado: se diseño e implementó una API REST con CRUD completo conectada a una base de datos relacional, se construyó una infraestructura de pruebas reproducible mediante Docker, se ejecutaron pruebas de carga y de estrés con simulación de usuarios concurrentes, y se reportaron tiempos de respuesta, throughput y tasas de error en ambos escenarios.

Comparativa carga vs estrés

La tabla siguiente sintetiza los resultados de ambos escenarios para facilitar su comparación:

Métrica	Prueba de carga (100 VUs)	Prueba de estrés (hasta 600 VUs)
Duración	11 minutos	18 minutos
Total de requests	84.800	327.686
Iteraciones CRUD	28.500	65.616
Throughput pico	45 req/s sostenido	240 req/s (colapso)
Latencia p95 máxima	160 ms (PUT)	30 segundos
Tasa de error HTTP	0,00 %	0,41 %
Conexiones BD pico	10-11 (pool saturado)	10-11 (pool saturado)
CPU pico API	0,25 cores	0,95 cores
Memoria pico API	60 MiB estable	165 MiB (sin liberar)
Resultado	Sistema saludable	Quiebre en 400 VUs

Hallazgos principales

- **El sistema cumple con la carga objetivo.** Bajo 100 VUs concurrentes durante 10 minutos no se reportaron errores HTTP, las latencias p95 se mantuvieron muy por debajo de los thresholds definidos y el throughput fue estable en 45 req/s.
- **Punto de quiebre identificado en 400 VUs.** A partir del escalón 4 de la prueba de estrés la latencia salta abruptamente de menos de 5 segundos a 15 segundos, evidencia inequívoca del quiebre del sistema.
- **El cuello de botella es el pool de conexiones de la API, no la BD.** Postgres operaba al 40 % de capacidad de CPU cuando el sistema colapsó. La API tenía toda la capacidad de procesar más requests pero estaban bloqueadas esperando una conexión libre del pool de 10.
- **La tasa de error HTTP subestima la degradación real.** Solo 0,41 % de las requests devolvieron error HTTP, pero la mayoría de las requests "exitosas" durante el estrés tomaron entre 15 y 30 segundos, tiempo funcionalmente equivalente a una falla desde la perspectiva del usuario.
- **Posible fuga de memoria menor en la API.** La memoria de la API no retorna al baseline tras la prueba de estrés, sino que se estabiliza en aproximadamente 160 MiB. Este comportamiento amerita investigación adicional con herramientas de profiling para descartar leak real.